

1. System architecture

Training fan-out — every client starts from the same immutable revision

Participant A	Model repository	Participant B
Private dataset A	<code>main @ C_r</code>	Private dataset B
↓ exact-revision download	model + config + round manifest	exact-revision download ↓
download → own trainer → upload or trusted local plugin	versioned coordination plane	download → own trainer → upload or trusted local plugin
local checkpoint →	submission A submission B	← local checkpoint

Aggregation fan-in — only the owner publishes the global model

Inputs	Repository owner	Output
Pinned A revision Pinned B revision	Validate uploader allowlist, base, round, hashes and tensor schema FedAvg + optional owner evaluation	Publish <code>C_(r+1)</code> to <code>main</code> Record manifest + optional tag

Hugging Face Hub uses PR refs; JFrog HuggingFaceML uses unique named revisions discovered with AQL. Neither backend is a privacy boundary: raw data stays local, but uploaded weights are visible to authorized readers.

2. Operation sequence

Step	Actor	Operation	Required guard
1	Owner	Run <code>python -m hf2l.init_repo</code> ; publish round 0	Record returned revision <code>c0</code>
2	Owner	Send repository ID and <code>c0</code> to A and B	Send immutable ID—not “latest”
3	A + B	Run <code>python -m hf2l.client_download</code> , then any local trainer	Write a separate compatible checkpoint
4	A + B	Run <code>python -m hf2l.client_upload</code>	Hub PR or unique JFrog revision; data stays local
5	Owner	Discover or explicitly select submissions	Allowlisted uploaders; current base + round
6	Owner	Dry-run validation, weighted FedAvg, optional evaluation	Reject stale, incompatible or non-finite updates
7	Owner	Publish aggregate and optional round tag/revision	Hub CAS or single JFrog coordinator

Round transition

`main @ c0` → parallel client submissions → owner aggregation → `main @ c1` → repeat

Hub provides atomic `parent_commit` protection. JFrog is rechecked before publication and requires one writer or an external coordinator lock.

3. Credential management

Actor	Capability	Credential policy
Owner: initialization	Create/initialize repository and first revision	Narrowly scoped owner write token
Participant	Read <code>c_r</code> ; upload an isolated submission	Each participant's own token; never the owner's token
Owner: aggregation	Read submissions; update only <code>main</code>	Separate repository-scoped write token
Automated publisher	Publish one repository from CI	Prefer a short-lived trusted-publisher token

Resolution used by the scripts

Hub: `--token` → `HF_TOKEN` → active `hf auth login` cache

JFrog: `--token` → `JFROG_ACCESS_TOKEN` → `HF_TOKEN`

- Interactive workstation: use `hf auth login` , then verify with `hf auth whoami` .
- JFrog: set `HF_ENDPOINT=.../api/huggingfaceml/REPO_KEY` ; create the local repository as an administrator and disable participant overwrite permission.
- Automation: inject tokens from a secret manager; mask logs and rotate them.
- Avoid `--token` for routine use because arguments may appear in history and process listings. Never commit tokens, `.env` files, or credential caches.
- Hub: enable PRs. JFrog: scope read/deploy rights by repository and identity.

4. Handling large models

The workflow supports one SafeTensors file or an indexed, deterministically sharded checkpoint. Aggregation processes one shard at a time.

Stage	Scalable design
Package	Deterministic sharded SafeTensors + index. All clients use the same architecture, names, dtypes and shard layout.
Transfer	Use exact revisions and a local cache. Hub supports Xet; current JFrog HuggingFaceML versions can enable Xet for files over 50 GB.
Aggregate	Process one shard/tensor from every accepted client, accumulate in FP32/FP64 on CPU, write the output shard, then release input memory.
Finalize	Validate hashes, config, index, tensor schemas and non-finite values. Publish the model plus a round manifest naming <code>base_revision=C_r</code> .

Resource profile: memory is bounded by one output shard plus active tensors and the FP32/FP64 accumulator; disk holds every selected snapshot and one output.

Xet uses content-defined chunks around 64 KiB, deduplicates known chunks, and adapts transfer concurrency. Dense training may alter bytes throughout every weight file, so deduplication can be limited. Deltas or adapters reduce traffic only when the protocol defines how they are reconstructed and aggregated.